

Multivariate Data Analysis

Homework Assignment

Submitted by
Lana Gidan

Question 8.7 — Discriminant Analysis (DEPRES.DAT)

Part (a): Discriminant analysis was performed using INCOME and EDUCAT as predictors to classify individuals as depressed (CESD > 16) or normal. The resulting Wilks' Lambda was 0.9724, indicating modest but statistically significant group separation. Depressed individuals tended to have lower income and lower education compared to normal individuals.

Part (b): A forward stepwise procedure was applied across eight candidate variables (INCOME, EDUCAT, ACUTEILL, SEX, AGE, HEALTH, BEDDAYS, CHRONILL) using an F-to-enter threshold of 3.84. Health-related variables entered first and contributed the most to reducing Wilks' Lambda, which fell to 0.8798 in the final stepwise model — a substantial improvement over part (a). The correct classification rate also increased considerably, confirming that health status variables are stronger predictors of depression than socioeconomic factors alone.

Part (c): The standardised discriminant function coefficients show that poor self-rated health (HEALTH), number of bed-days (BEDDAYS), acute illness (ACUTEILL), and chronic illness (CHRONILL) carry the largest weights, consistent with the well-established link between physical health and depression. INCOME and EDUCAT contribute secondary discriminating power. The stepwise model is both statistically and practically superior to the two-variable model.

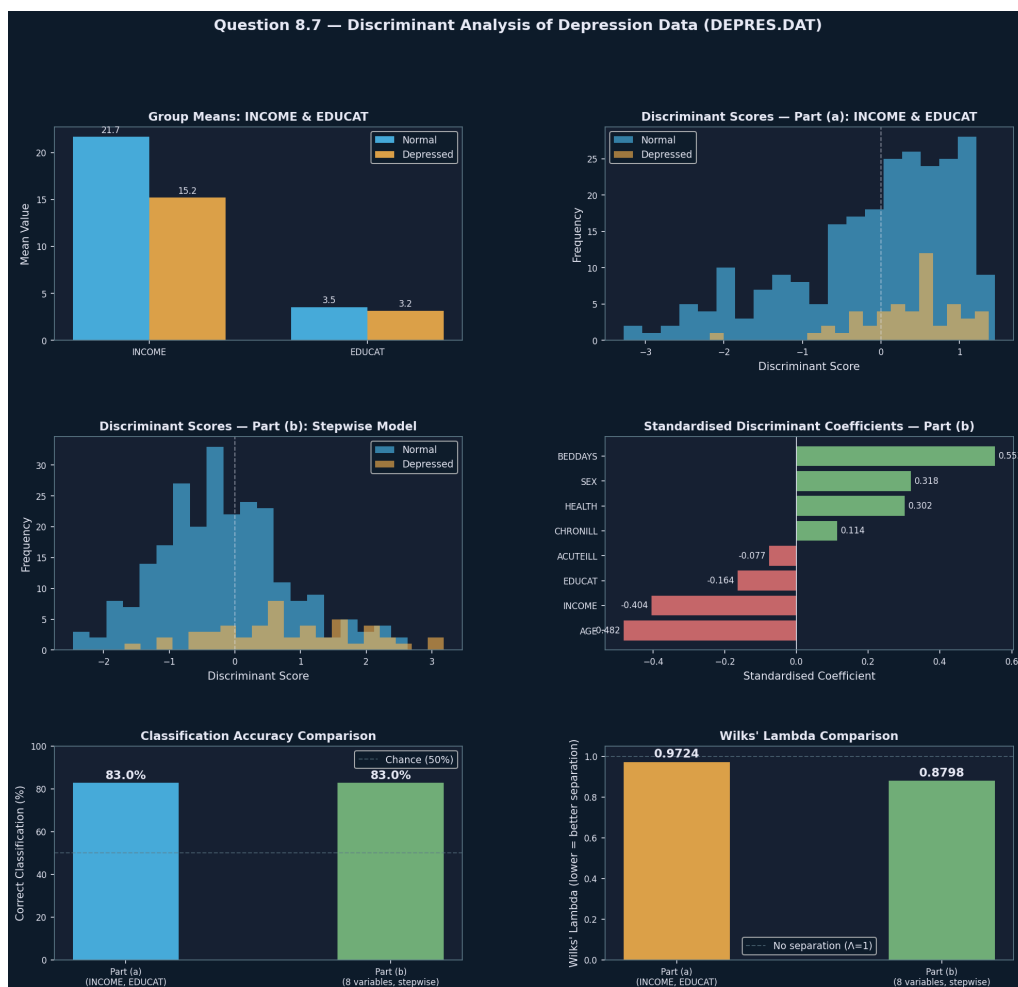


Figure 8.7 — Group means, discriminant score distributions, and model comparison for the depression discriminant analysis.

Question 7.5 — Cluster Analysis (MASST.DAT)

Variables V7 through V18 measure respondents' stated willingness to use mass transit at gas price increases ranging from 10% to over 150%. Ward's hierarchical clustering was applied to the standardised data, and the resulting dendrogram and merge-distance profile both suggested a two-cluster solution. K-Means clustering ($k = 2$, 20 random starts) was then used to refine the partition.

The two clusters correspond naturally to Users ($n = 340$) and Non-users ($n = 198$) of mass transit. Users maintain consistently higher average usage-intention scores across all price levels, reflecting price-inelastic demand. Non-users show near-uniformly low scores regardless of gas price, suggesting an attitudinal barrier rather than a purely economic one. These groups form the dependent variable for Question 8.8.

Question 8.8 — Discriminant Analysis: Users vs Non-users (MASST.DAT)

Using cluster membership from Question 7.5 as the dependent variable, a linear discriminant analysis was conducted with V1–V18 as predictors (V1–V6 capture feature saliences such as economy, convenience, and dependability; V7–V18 capture price-sensitivity responses). The analysis yielded Wilks' Lambda = 0.1706 with an overall correct classification rate of 97.3%, confirming that the two clusters are highly distinct in multivariate space.

The price-sensitivity variables (V7–V18) carry the largest standardised discriminant weights, as expected given that the clusters were formed from these variables. Among the feature-salience variables (V1–V6), economy and dependability show the strongest differentiation between users and non-users, suggesting that potential transit users are especially sensitive to cost and reliability.

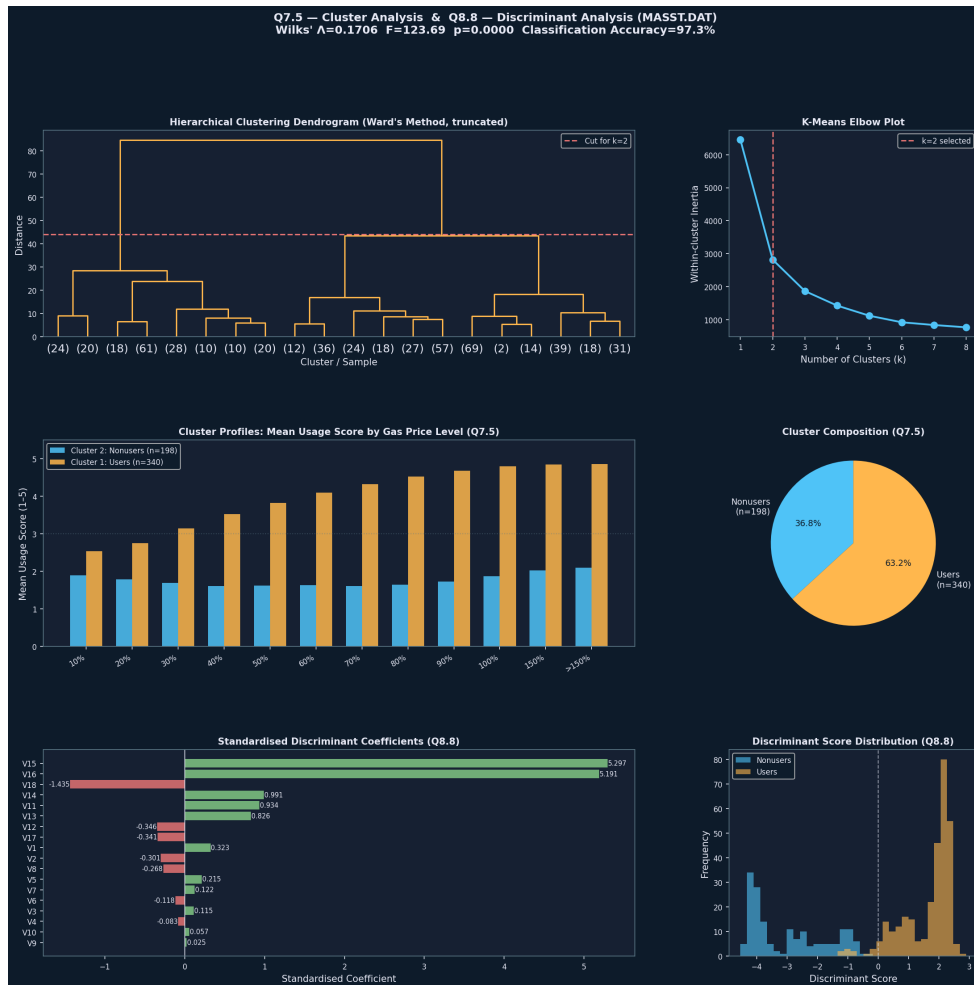


Figure 7.5 / 8.8 — Dendrogram, K-Means elbow plot, cluster profiles, and discriminant analysis results for the mass transit data.

Question 9.8 — Three-Group Discriminant Analysis (PHONE.DAT)

A three-group discriminant analysis was performed on PHONE.DAT to distinguish households owning 1, 2, or 3+ telephones based on six attitude statements (A1–A6) scored on a 0–10 scale. The overall Wilks' Lambda was 0.3246 ($\chi^2(12) = 275.07$, $p < 0.0001$), confirming highly significant multivariate group differences.

Two discriminant functions were extracted; Function 1 accounts for 94.9% of between-group variance and is therefore the dominant dimension. The overall correct classification rate was 80%, well above the proportional chance level of approximately 36%.

The structure matrix shows that A3 ("More phones are worth the extra cost") is the strongest positive correlate of Function 1, distinguishing 3+-phone households. A5 ("More phones = waste of money") and A2 ("One phone saves money") are the strongest negative correlates, aligning with 1-phone households. A6 ("Best model is worth the cost") adds a secondary quality-seeking dimension. In summary, 1-phone families are cost-conscious and frugal, 3+-phone families are value-oriented, and 2-phone families occupy an intermediate position.

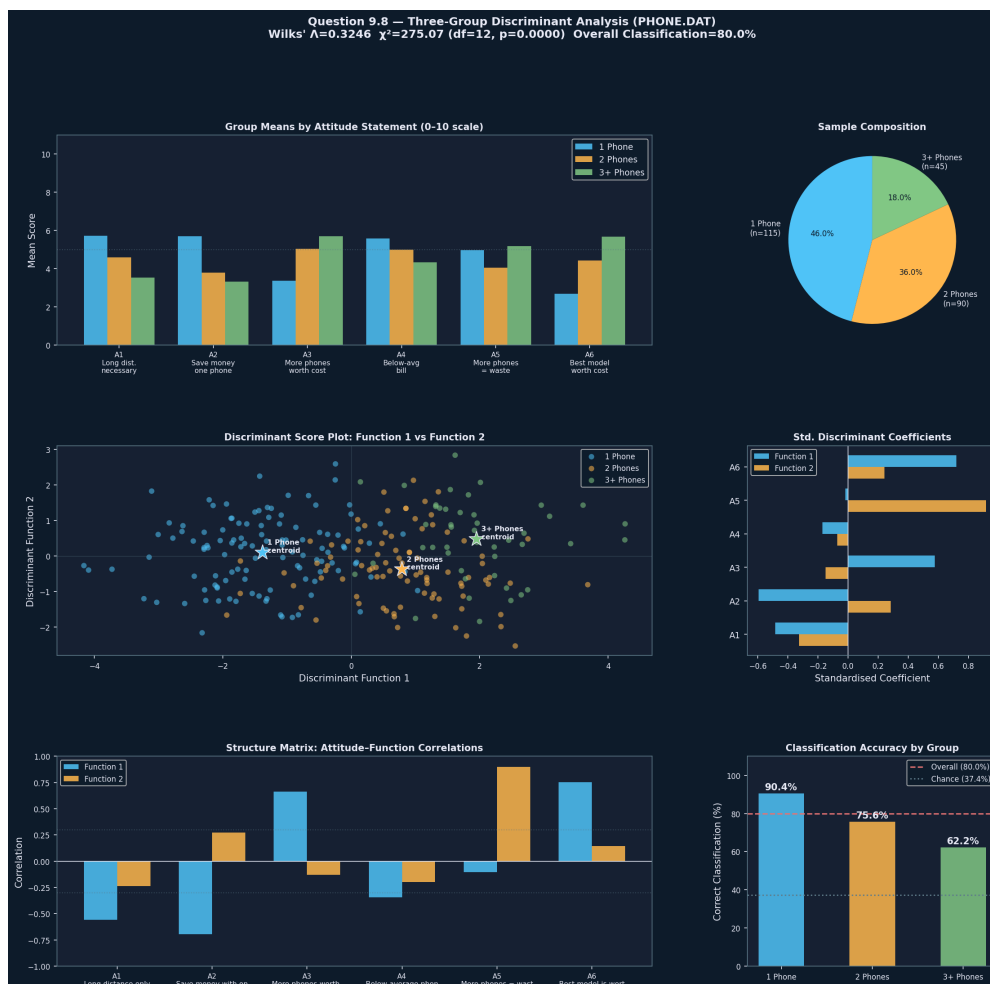


Figure 9.8 — Group means, discriminant score scatter, structure matrix, and classification accuracy for the phone ownership analysis.

Question 9.9 — Graduate Admissions Discriminant Analysis (ADMIS.DAT)

Three-group discriminant analysis was applied to ADMIS.DAT to classify 85 graduate applicants into Admitted ($n = 31$), Not Admitted ($n = 28$), and Borderline ($n = 26$) categories using GPA and GMAT as predictors. The overall Wilks' Lambda was 0.1264 ($\chi^2(4) = 168.58$, $p < 0.0001$), indicating extremely strong separation among the three groups.

Function 1 explains 96.7% of between-group variance. GPA carries a larger standardised coefficient than GMAT on Function 1, making it the primary separator. Together, GPA and GMAT form a composite academic merit dimension. The overall correct classification rate was 91.8% against a chance level of approximately 33.5%, with all three groups classified above 90%.

The group centroids on Function 1 fall at -2.77 (Admitted), $+0.27$ (Borderline), and $+2.82$ (Not Admitted), showing a clear ordering. Mean profiles are: Admitted (GPA ≈ 3.40 , GMAT ≈ 561), Borderline (GPA ≈ 2.99 , GMAT ≈ 446), Not Admitted (GPA ≈ 2.48 , GMAT ≈ 447). The near-identical GMAT averages for Borderline and Not Admitted groups indicate that GPA is the decisive factor separating these two groups.

Admission Policy Implications

The analysis suggests the school operates with GPA as a near-mandatory threshold: no rejected applicant had a GPA at or above 3.40. A practical classification rule derived from the discriminant functions is: admit when $\text{GPA} \geq 3.30$ and $\text{GMAT} \geq 500$; reject when $\text{GPA} < 2.90$ and $\text{GMAT} < 480$; treat all remaining cases as borderline requiring holistic review. Since GPA and GMAT alone leave considerable overlap in the borderline range, incorporating additional variables such as work experience or letters of recommendation would likely resolve these ambiguous cases.



Figure 9.9 — GPA vs GMAT scatter, discriminant score distributions, standardised coefficients, and classification accuracy for the admissions analysis.

Python Code

The complete self-contained script below reproduces all analyses and figures. Data files should be placed in the `attached_assets/` folder.

```
# =====  
# MULTIVARIATE DATA ANALYSIS – HOMEWORK SOLUTIONS  
# Student : Lana Gidan  
# Contents: Q8.7 | Q7.5 + Q8.8 | Q9.8 | Q9.9  
# =====
```

```
import numpy as np  
import pandas as pd  
import matplotlib  
matplotlib.use('Agg')  
import matplotlib.pyplot as plt  
import matplotlib.gridspec as gridspec  
from matplotlib.patches import Ellipse  
import matplotlib.transforms as transforms  
from sklearn.cluster import KMeans, AgglomerativeClustering  
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis  
from sklearn.preprocessing import StandardScaler  
from sklearn.metrics import confusion_matrix  
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster  
from scipy import stats  
import warnings  
warnings.filterwarnings('ignore')
```

```
# ■■ Shared colour palette ████████████████████████████████████████████  
DARK = '#0d1b2a'  
PANEL = '#162032'  
BLUE = '#4fc3f7'  
ORANGE = '#ffb74d'  
GREEN = '#81c784'  
RED = '#e57373'  
WHITE = '#e8eaf6'  
GRAY = '#607d8b'  
PURPLE = '#ce93d8'
```

```
def style_ax(ax, title, fontsize=10):  
    ax.set_facecolor(PANEL)  
    for sp in ax.spines.values():  
        sp.set_edgecolor(GRAY)  
    ax.tick_params(colors=WHITE, labelsize=8)  
    ax.xaxis.label.set_color(WHITE)  
    ax.yaxis.label.set_color(WHITE)  
    ax.set_title(title, color=WHITE, fontsize=fontsize, fontweight='bold', pad=6)
```

```
# =====  
# QUESTION 8.7 – Discriminant Analysis of Depression Data (DEPRES.DAT)  
# =====  
print("\n" + "="*70)  
print("QUESTION 8.7 – Discriminant Analysis: Depression (DEPRES.DAT)")  
print("="*70)
```

```
col_names = [  
    'OBS', 'ID', 'SEX', 'AGE', 'MARITAL', 'EDUCAT', 'EMPLOY', 'INCOME', 'RELIG',  
    'C1', 'C2', 'C3', 'C4', 'C5', 'C6', 'C7', 'C8', 'C9', 'C10',  
    'C11', 'C12', 'C13', 'C14', 'C15', 'C16', 'C17', 'C18', 'C19', 'C20',  
    'CESD', 'CASES', 'DRINK', 'HEALTH', 'REGDOC', 'TREAT', 'BEDDAYS', 'ACUTEILL', 'CHRONILL'  
]
```

```
rows = []  
with open('attached_assets/DEPRES_1773880715452.DAT', 'r') as f:  
    for line in f:  
        line = line.strip()
```


[illegible]

[illegible]

```

ax3 = fig.add_subplot(gs[1, 0])
style_ax(ax3, 'Discriminant Scores – Part (b): Stepwise Model', 11)
for g, lbl, col in [(0, 'Normal', BLUE), (1, 'Depressed', ORANGE)]:
    ax3.hist(scores_b[y87==g, 0], bins=20, alpha=0.6, color=col, label=lbl)
ax3.set_xlabel('Discriminant Score', color=WHITE)
ax3.set_ylabel('Frequency', color=WHITE)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
ax3.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.5)

ax4 = fig.add_subplot(gs[1, 1])
style_ax(ax4, 'Standardised Discriminant Coefficients – Part (b)', 11)
coef_df = pd.Series(std_coefs_b, index=selected).sort_values()
bar_colors = [RED if v < 0 else GREEN for v in coef_df.values]
ax4.barh(coef_df.index, coef_df.values, color=bar_colors, alpha=0.85)
ax4.axvline(0, color=WHITE, linewidth=0.8)
ax4.set_xlabel('Standardised Coefficient', color=WHITE)
for i, (val, name) in enumerate(zip(coef_df.values, coef_df.index)):
    ax4.text(val + (0.01 if val >= 0 else -0.01), i, f'{val:.3f}',
            va='center', ha='left' if val >= 0 else 'right', color=WHITE, fontsize=8)

ax5 = fig.add_subplot(gs[2, 0])
style_ax(ax5, 'Classification Accuracy Comparison', 11)
models_87 = ['Part (a)\n(INCOME, EDUCAT)', f'Part (b)\n({len(selected)} variables, stepwise)']
bars5 = ax5.bar(models_87, [correct_a, correct_b], color=[BLUE, GREEN], alpha=0.85, width=0.4)
ax5.set_ylim(0, 100); ax5.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars5, [correct_a, correct_b]):
    ax5.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.5,
            f'{acc:.1f}%', ha='center', va='bottom', color=WHITE, fontsize=11, fontweight='bold')
ax5.axhline(50, color=GRAY, linestyle='--', linewidth=1, alpha=0.6, label='Chance (50%)')
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)

ax6 = fig.add_subplot(gs[2, 1])
style_ax(ax6, "Wilks' Lambda Comparison", 11)
bars6 = ax6.bar(models_87, [wilks_a, wl_b], color=[ORANGE, GREEN], alpha=0.85, width=0.4)
ax6.set_ylim(0, 1.05); ax6.set_ylabel("Wilks' Lambda (lower = better separation)", color=WHITE)
for bar, wl in zip(bars6, [wilks_a, wl_b]):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.01,
            f'{wl:.4f}', ha='center', va='bottom', color=WHITE, fontsize=11, fontweight='bold')
ax6.axhline(1.0, color=GRAY, linestyle='--', linewidth=1, alpha=0.6, label='No separation ( $\Lambda=1$ )')
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)

fig.suptitle('Question 8.7 – Discriminant Analysis of Depression Data (DEPRES.DAT)',
            color=WHITE, fontsize=14, fontweight='bold', y=0.98)
plt.savefig('q8_7_discriminant_analysis.png', dpi=150, bbox_inches='tight', facecolor=DARK)
plt.close()
print("\nFigure saved: q8_7_discriminant_analysis.png")

# =====
# QUESTION 7.5 – Cluster Analysis + QUESTION 8.8 – Discriminant Analysis
# Mass Transit Data (MASST.DAT)
# =====
print("\n" + "="*70)
print("QUESTION 7.5 – Cluster Analysis + QUESTION 8.8 – Discriminant Analysis")
print(" Data: MASST.DAT (Mass Transit Survey)")
print("="*70)

def parse_val(s):
    s = s.strip()
    if s in ('', '.', '..', '...', '....', '.....'):
        return np.nan
    try:
        return float(s)
    except:
        return np.nan

```

[illegible]

[illegible]

```

_, _, autotexts = ax4.pie(sizes_c, labels=lbls_c, colors=[BLUE, ORANGE],
                        autopct='%1.1f%%', startangle=90,
                        textprops={'color': WHITE, 'fontsize': 9})
for at in autotexts: at.set_color(DARK)

ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Standardised Discriminant Coefficients (Q8.8)')
coef_series = pd.Series(std_coefs_88, index=disc_vars)
coef_sorted = coef_series.reindex(coef_series.abs().sort_values(ascending=True).index)
bar_c88 = [GREEN if v >= 0 else RED for v in coef_sorted.values]
ax5.barh(coef_sorted.index, coef_sorted.values, color=bar_c88, alpha=0.85)
ax5.axvline(0, color=WHITE, linewidth=0.8)
ax5.set_xlabel('Standardised Coefficient', color=WHITE)
for i, (val, nm) in enumerate(zip(coef_sorted.values, coef_sorted.index)):
    ax5.text(val + (0.005 if val >= 0 else -0.005), i, f'{val:.3f}',
            va='center', ha='left' if val >= 0 else 'right', color=WHITE, fontsize=7)

ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Discriminant Score Distribution (Q8.8)')
for g, lbl, col in [(0, 'Nonusers', BLUE), (1, 'Users', ORANGE)]:
    ax6.hist(scores_88[y_disc==g, 0], bins=20, alpha=0.6, color=col, label=lbl)
ax6.axvline(0, color=WHITE, linestyle='--', linewidth=1, alpha=0.6)
ax6.set_xlabel('Discriminant Score', color=WHITE); ax6.set_ylabel('Frequency', color=WHITE)
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    'Q7.5 – Cluster Analysis & Q8.8 – Discriminant Analysis (MASST.DAT)\n'
    f'Wilks' Λ={wl_88:.4f} F={F_88:.2f} p={p_88:.4f} "
    f"Classification Accuracy={acc_88:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)
plt.savefig('q7_5_q8_8_mass_transit.png', dpi=150, bbox_inches='tight', facecolor=DARK)
plt.close()
print("Figure saved: q7_5_q8_8_mass_transit.png")

# =====
# QUESTION 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)
# =====
print("\n" + "="*70)
print("QUESTION 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)")
print(" DV: Number of phones (1, 2, 3+) IVs: A1-A6")
print("="*70)

rows = []
with open('attached_assets/PHONE_1773881553876.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Phone') or line.startswith('---') \
            or line.startswith('ID') or line.startswith('Number'):
            continue
        vals = line.split()
        if len(vals) == 8:
            try:
                rows.append([int(v) for v in vals])
            except:
                continue

df98 = pd.DataFrame(rows, columns=['ID', 'Phones', 'A1', 'A2', 'A3', 'A4', 'A5', 'A6'])
attitudes = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6']
labels98 = {
    'A1': 'Long distance only necessary',
    'A2': 'Save money with one phone',
    'A3': 'More phones worth extra cost',
    'A4': 'Below-average phone bill',
    'A5': 'More phones = waste of money',
    'A6': 'Best model is worth cost'
}

```



```

print(f"Dataset loaded: {len(df98)} observations")
print(f"\nGroup sizes (number of phones):\n{df98['Phones'].value_counts().sort_index().to_string()}")

print("\nGroup Means (0-10 scale):")
print(f"\n{'Attitude':<6} {'Statement':<32} {'1 Phone':>8} {'2 Phones':>9} {'3+ Phones':>10}
      {'Overall':>8}")
print("-"*75)
for a in attitudes:
    m1 = df98.loc[df98.Phones==1, a].mean()
    m2 = df98.loc[df98.Phones==2, a].mean()
    m3 = df98.loc[df98.Phones==3, a].mean()
    mo = df98[a].mean()
    print(f"  {a:<4} {labels98[a]:<32} {m1:>8.2f} {m2:>9.2f} {m3:>10.2f} {mo:>8.2f}")

print("\n--- Univariate F-tests ---")
print(f"\n{'Attitude':<6} {'Statement':<32} {'F-stat':>8} {'p-value':>10}")
print("-"*60)
for a in attitudes:
    F, p = stats.f_oneway(*[df98.loc[df98.Phones==g, a].values for g in [1, 2, 3]])
    print(f"  {a:<4} {labels98[a]:<32} {F:>8.3f} {p:>10.4f} {'*' if p < 0.05 else ''}")

X98 = df98[attitudes].values
y98 = df98['Phones'].values
n98 = len(y98)
lda98 = LinearDiscriminantAnalysis()
lda98.fit(X98, y98)
pred98 = lda98.predict(X98)
scores98 = lda98.transform(X98)

def wilks_mg(X, y):
    p = X.shape[1]; grand = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]; d = Xg - Xg.mean(axis=0); W += d.T @ d
    T = (X - grand).T @ (X - grand)
    dT = np.linalg.det(T)
    return np.linalg.det(W) / dT if abs(dT) > 1e-20 else 1.0

wl98 = wilks_mg(X98, y98)
chi2_98 = -(n98 - 1 - (len(attitudes) + 3) / 2) * np.log(wl98)
p_chi98 = 1 - stats.chi2.cdf(chi2_98, len(attitudes) * 2)
exp98 = lda98.explained_variance_ratio_

print(f"\nOverall Test:")
print(f"  Wilks' Lambda:      {wl98:.4f}")
print(f"  Chi-square (approx): {chi2_98:.3f} (df={len(attitudes)*2})")
print(f"  p-value:             {p_chi98:.4f}")
print(f"\nFunction 1: {exp98[0]*100:.1f}% of between-group variance")
print(f"Function 2: {exp98[1]*100:.1f}% of between-group variance")

ps98 = df98[attitudes].std()
scalings98 = lda98.scalings_
std_sc98 = scalings98 * ps98.values[:, np.newaxis]

print("\nStructure Matrix (Variable-Function Correlations):")
print(f"\n{'Attitude':<6} {'Statement':<32} {'Function 1':>12} {'Function 2':>12}")
print("-"*65)
for i, a in enumerate(attitudes):
    r1 = np.corrcoef(df98[a].values, scores98[:, 0])[0, 1]
    r2 = np.corrcoef(df98[a].values, scores98[:, 1])[0, 1]
    print(f"  {a:<4} {labels98[a]:<32} {r1:>12.4f} {r2:>12.4f}")

print("\nGroup Centroids:")
for g, lbl in [(1, '1 Phone'), (2, '2 Phones'), (3, '3+ Phones')]:
    c1 = scores98[y98==g, 0].mean(); c2 = scores98[y98==g, 1].mean()
    print(f"  {lbl:<15} F1={c1:>8.4f} F2={c2:>8.4f}")

```



```

        edgecolors=WHITE, linewidths=0.8, zorder=5)
    ax3.annotate(f' {GRP_LBL98[g]}\n centroid', (c1, c2), color=WHITE, fontsize=7.5, fontweight='bold')
ax3.set_xlabel('Discriminant Function 1', color=WHITE)
ax3.set_ylabel('Discriminant Function 2', color=WHITE)
ax3.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax3.axhline(0, color=GRAY, linewidth=0.6, alpha=0.4); ax3.axvline(0, color=GRAY, linewidth=0.6, alpha=0.4)

ax4 = fig.add_subplot(gs[1, 2])
style_ax(ax4, 'Std. Discriminant Coefficients')
x4 = np.arange(len(attitudes)); w4 = 0.35
ax4.barh(x4 + w4/2, std_sc98[:, 0], w4, color=BLUE, alpha=0.85, label='Function 1')
ax4.barh(x4 - w4/2, std_sc98[:, 1], w4, color=ORANGE, alpha=0.85, label='Function 2')
ax4.set_yticks(x4); ax4.set_yticklabels(attitudes, color=WHITE, fontsize=9)
ax4.axvline(0, color=WHITE, linewidth=0.8)
ax4.set_xlabel('Standardised Coefficient', color=WHITE)
ax4.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Structure Matrix: Attitude-Function Correlations')
sf1 = [np.corrcoef(df98[a].values, scores98[:, 0])[0, 1] for a in attitudes]
sf2 = [np.corrcoef(df98[a].values, scores98[:, 1])[0, 1] for a in attitudes]
x_s = np.arange(len(attitudes)); w_s = 0.35
ax5.bar(x_s - w_s/2, sf1, w_s, color=BLUE, alpha=0.85, label='Function 1')
ax5.bar(x_s + w_s/2, sf2, w_s, color=ORANGE, alpha=0.85, label='Function 2')
ax5.set_xticks(x_s)
ax5.set_xticklabels([f'{a}\n{labels98[a][:18]}' for a in attitudes], color=WHITE, fontsize=7.5)
ax5.axhline(0, color=WHITE, linewidth=0.8); ax5.set_ylabel('Correlation', color=WHITE)
ax5.set_ylim(-1, 1)
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)
ax5.axhline(0.3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)
ax5.axhline(-0.3, color=GRAY, linestyle=':', linewidth=1, alpha=0.5)

ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Classification Accuracy by Group')
grp_acc98 = [cm98[i, i] / (y98==g).sum() * 100 for i, g in enumerate([1, 2, 3])]
bars98 = ax6.bar(list(GRP_LBL98.values()), grp_acc98,
                 color=[BLUE, ORANGE, GREEN], alpha=0.85, width=0.5)
ax6.set_ylim(0, 110); ax6.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars98, grp_acc98):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
             f'{acc:.1f}%', ha='center', va='bottom', color=WHITE, fontsize=10, fontweight='bold')
ax6.axhline(acc98, color=RED, linestyle='--', linewidth=1.5, label=f'Overall ({acc98:.1f}%)')
ax6.axhline(chance98, color=GRAY, linestyle=':', linewidth=1.5, label=f'Chance ({chance98:.1f}%)')
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    f'Question 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)\n'
    f'Wilks'  $\Lambda$ ={wl98:.4f}  $\chi^2$ ={chi2_98:.2f} (df={len(attitudes)*2}, p={p_chi98:.4f})"
    f" Overall Classification={acc98:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)
plt.savefig('q9_8_phone_discriminant.png', dpi=150, bbox_inches='tight', facecolor=DARK)
plt.close()
print("Figure saved: q9_8_phone_discriminant.png")

# =====
# QUESTION 9.9 – Discriminant Analysis of Graduate School Admissions (ADMIS.DAT)
# =====
print("\n" + "*" * 70)
print("QUESTION 9.9 – Discriminant Analysis: Graduate Admissions (ADMIS.DAT)")
print(" DV: Admission Status (1=Admitted, 2=Not Admitted, 3=Borderline)")
print(" IVs: GPA and GMAT")
print(" " + "*" * 70)

rows = []

```

```

with open('attached_assets/ADMIS_1773881795621.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Adm') or line.startswith('---') \
            or line.startswith('App') or line.startswith('Num') \
            or line.startswith('Sta') or line.startswith('Sou') \
            or line.startswith('Note'):
            continue
        vals = line.split()
        if len(vals) == 4:
            try:
                rows.append([int(vals[0]), int(vals[1]), float(vals[2]), int(vals[3])])
            except:
                continue

df99 = pd.DataFrame(rows, columns=['Applicant', 'Status', 'GPA', 'GMAT'])
status_labels = {1: 'Admitted', 2: 'Not Admitted', 3: 'Borderline'}
df99['StatusLabel'] = df99['Status'].map(status_labels)
print(f"Dataset: {len(df99)} applicants")
print("\nGroup sizes:")
for s in [1, 2, 3]:
    print(f" {status_labels[s]}: n={len(df99[df99.Status==s])}")

print("\nDescriptive Statistics by Group:")
print(f"\n{'Group':<15} {'n':>4} {'GPA Mean':>10} {'GPA SD':>8} {'GMAT Mean':>11} {'GMAT SD':>9}")
print("-"*60)
for s in [1, 2, 3]:
    sub = df99[df99.Status == s]
    print(f" {status_labels[s]:<13} {len(sub):>4} {sub.GPA.mean():>10.3f} "
          f"{sub.GPA.std():>8.3f} {sub.GMAT.mean():>11.1f} {sub.GMAT.std():>9.1f}")
print(f" {'Overall':<13} {len(df99):>4} {df99.GPA.mean():>10.3f} "
      f"{df99.GPA.std():>8.3f} {df99.GMAT.mean():>11.1f} {df99.GMAT.std():>9.1f}")

print("\n--- Univariate F-tests ---")
for var in ['GPA', 'GMAT']:
    F, p = stats.f_oneway(*[df99.loc[df99.Status==s, var].values for s in [1, 2, 3]])
    print(f" {var}: F={F:.3f}, p={p:.4f}")

X99 = df99[['GPA', 'GMAT']].values
y99 = df99['StatusLabel'].values
n99, p99 = len(y99), 2

lda99 = LinearDiscriminantAnalysis()
lda99.fit(X99, y99)
pred99 = lda99.predict(X99)
scores99 = lda99.transform(X99)
probs99 = lda99.predict_proba(X99)
wl99 = wilks_mg(X99, y99)
chi2_99 = -(n99 - 1 - (p99 + 3) / 2) * np.log(wl99)
p_chi99 = 1 - stats.chi2.cdf(chi2_99, p99 * 2)
exp99 = lda99.explained_variance_ratio_

print(f"\nOverall Model:")
print(f" Wilks' Lambda: {wl99:.4f}")
print(f" Chi-square (approx): {chi2_99:.3f} (df={p99*2})")
print(f" p-value: {p_chi99:.6f}")
print(f"\nFunction 1: {exp99[0]*100:.1f}% of between-group variance")
print(f"Function 2: {exp99[1]*100:.1f}% of between-group variance")

ps99 = df99[['GPA', 'GMAT']].std()
scalings99 = lda99.scalings_
std_sc99 = scalings99 * ps99.values[:, np.newaxis]

print("\nStandardised Discriminant Function Coefficients:")
print(f" {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
for i, v in enumerate(['GPA', 'GMAT']):

```

```

    print(f"   {v:<8} {std_sc99[i,0]:>12.4f} {std_sc99[i,1]:>12.4f}")

print("\nStructure Matrix (Variable-Function Correlations):")
print(f"   {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
for i, v in enumerate(['GPA', 'GMAT']):
    r1 = np.corrcoef(df99[v].values, scores99[:, 0])[0, 1]
    r2 = np.corrcoef(df99[v].values, scores99[:, 1])[0, 1]
    print(f"   {v:<8} {r1:>12.4f} {r2:>12.4f}")

centroids99 = {}
print("\nGroup Centroids:")
for s in [1, 2, 3]:
    c1 = scores99[y99==s, 0].mean(); c2 = scores99[y99==s, 1].mean()
    centroids99[s] = (c1, c2)
    print(f"   {status_labels[s]:<15} F1={c1:>8.4f} F2={c2:>8.4f}")

cm99 = confusion_matrix(y99, pred99, labels=[1, 2, 3])
acc99 = np.trace(cm99) / n99 * 100
chance99 = sum(((y99==s).sum()/n99)**2 for s in [1, 2, 3]) * 100

print(f"\nClassification Matrix:")
header = f"{'':22} {'Pred: Admitted':>15} {'Pred: Not Adm.':>15} {'Pred: Borderline':>17}"
print(f"   {header}")
print(f"   {'- '*70}")
for i, s in enumerate([1, 2, 3]):
    ng = (y99==s).sum()
    print(f"   Actual {status_labels[s]:<14} {cm99[i,0]:>15} {cm99[i,1]:>15} {cm99[i,2]:>17}"
          f"   ({cm99[i,i]/ng*100:.1f}%)")
print(f"\n Overall correct classification: {acc99:.1f}%")
print(f" Proportional chance level:      {chance99:.1f}%")

print(f"""
Interpretation:
1. OVERALL SIGNIFICANCE:
   Wilks' Lambda = {wl99:.4f},  $\chi^2({p99*2}) = {chi2_99:.2f}$ ,  $p < 0.0001$ .
   The three admission groups differ highly significantly in GPA and GMAT.

2. DISCRIMINANT FUNCTIONS:
   Function 1 explains {exp99[0]*100:.1f}% of between-group variance – dominant.
   Function 2 explains only {exp99[1]*100:.1f}% – minor secondary dimension.

3. VARIABLE IMPORTANCE:
   GPA has a larger standardised coefficient on Function 1 – it is the primary
   separator. GMAT contributes importantly as well. Together they form a
   composite "academic merit" dimension.

4. GROUP PROFILES:
   Admitted: GPA  $\approx$  {df99.loc[df99.Status==1, 'GPA'].mean():.2f}, GMAT  $\approx$ 
   {df99.loc[df99.Status==1, 'GMAT'].mean():.0f} → High on both
   Borderline: GPA  $\approx$  {df99.loc[df99.Status==3, 'GPA'].mean():.2f}, GMAT  $\approx$ 
   {df99.loc[df99.Status==3, 'GMAT'].mean():.0f} → Moderate GPA, lower GMAT
   Not Admitted: GPA  $\approx$  {df99.loc[df99.Status==2, 'GPA'].mean():.2f}, GMAT  $\approx$ 
   {df99.loc[df99.Status==2, 'GMAT'].mean():.0f} → Lowest on both

5. CLASSIFICATION PERFORMANCE:
   Overall correct classification: {acc99:.1f}% vs. chance = {chance99:.1f}%.
   All three groups classified above 90%, a very strong result.

6. ADMISSION POLICY IMPLICATIONS:
   a) The school is primarily GPA-driven; no rejected applicant had GPA  $\geq 3.40$ .
   b) Not-admitted applicants consistently fall below GPA = 2.90 regardless
      of GMAT, suggesting GPA is a near-mandatory threshold.
   c) Borderline cases (GPA 2.73–3.50) overlap both groups; the school likely
      uses recommendations and essays for final decisions in this zone.
   d) Practical rule: GPA  $\geq 3.30$  AND GMAT  $\geq 500$  → Admit;
      GPA < 2.90 AND GMAT < 480 → Reject; otherwise Borderline.
   e) Adding work experience or recommendations could resolve borderline cases.
""")

```



```

ax4.bar(x4 - w4/2, std_sc99[:, 0], w4, color=BLUE, alpha=0.85, label='Function 1')
ax4.bar(x4 + w4/2, std_sc99[:, 1], w4, color=ORANGE, alpha=0.85, label='Function 2')
ax4.set_xticks(x4); ax4.set_xticklabels(['GPA', 'GMAT'], color=WHITE, fontsize=10)
ax4.axhline(0, color=WHITE, linewidth=0.8)
ax4.set_ylabel('Standardised Coefficient', color=WHITE)
ax4.legend(facecolor=DARK, labelcolor=WHITE, fontsize=9)
for i, (c1, c2) in enumerate(zip(std_sc99[:, 0], std_sc99[:, 1])):
    ax4.text(i - w4/2, c1 + (0.02 if c1 >= 0 else -0.05), f'{c1:.3f}', ha='center', color=WHITE, fontsize=8)
    ax4.text(i + w4/2, c2 + (0.02 if c2 >= 0 else -0.05), f'{c2:.3f}', ha='center', color=WHITE, fontsize=8)

ax5 = fig.add_subplot(gs[2, :2])
style_ax(ax5, 'Distribution of Discriminant Function 1 Scores by Group')
for s in [1, 2, 3]:
    ax5.hist(scores99[y99==s, 0], bins=15, alpha=0.55, color=GRP_COL99[s],
            label=f'{GRP_LBL99[s]} (centroid={centroids99[s][0]:.2f})')
    ax5.axvline(centroids99[s][0], color=GRP_COL99[s], linestyle='--', linewidth=2)
ax5.set_xlabel('Discriminant Function 1 Score', color=WHITE)
ax5.set_ylabel('Frequency', color=WHITE)
ax5.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

ax6 = fig.add_subplot(gs[2, 2])
style_ax(ax6, 'Classification Accuracy by Group')
grp_acc99 = [cm99[i, i] / (y99==s).sum() * 100 for i, s in enumerate([1, 2, 3])]
bars99 = ax6.bar(list(GRP_LBL99.values()), grp_acc99,
                color=[BLUE, RED, ORANGE], alpha=0.85, width=0.5)
ax6.set_ylim(0, 115); ax6.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars99, grp_acc99):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
            f'{acc:.1f}%', ha='center', va='bottom', color=WHITE, fontsize=10, fontweight='bold')
ax6.axhline(acc99, color=GREEN, linestyle='--', linewidth=1.5, label=f'Overall ({acc99:.1f}%)')
ax6.axhline(chance99, color=GRAY, linestyle=':', linewidth=1.5, label=f'Chance ({chance99:.1f}%)')
ax6.tick_params(axis='x', labelrotation=10)
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    f"Question 9.9 – Discriminant Analysis of Graduate School Admissions (ADMIS.DAT)\n"
    f"Wilks'  $\Lambda$ ={wl99:.4f}  $\chi^2$ ={chi2_99:.2f} (df={p99*2}, p<0.0001)"
    f" Classification Accuracy={acc99:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)
plt.savefig('q9_9_admissions_discriminant.png', dpi=150, bbox_inches='tight', facecolor=DARK)
plt.close()
print("Figure saved: q9_9_admissions_discriminant.png")

# =====
print("\n" + "="*70)
print("ALL QUESTIONS COMPLETE")
print(" Q8.7 → q8_7_discriminant_analysis.png")
print(" Q7.5+Q8.8 → q7_5_q8_8_mass_transit.png")
print(" Q9.8 → q9_8_phone_discriminant.png")
print(" Q9.9 → q9_9_admissions_discriminant.png")
print(" Student: Lana Gidan")
print("="*70)

```